BIRLA INSTITUTE OF TECHNOLOGY, MESRA

Ranchi, Jharkhand

CLOUD COMPUTING LAB

Day to Day Performance: Project Report

Forecasting VM Resource Requirements for Proactive Provisioning

A Machine Learning Workload Predictor Integrated with CloudSim and a Monitoring Dashboard

(Question 10)

Submitted by

Name	Roll Number
Aditya Kumar	BTECH/10259/23
Anish Mehta	BTECH/10242/23
Tanishq Karmali	BTECH/10258/23

Course: Cloud Computing Lab

Institute: BIT Mesra

Submission deadline: 13th October 2026

Abstract

A cloud provider must decide how many virtual machines (VMs) to keep running before the demand for them arrives. A new VM takes minutes to become ready, so an autoscaler that only reacts to the current load is always a little late. Proactive provisioning tries to close this gap by forecasting demand and starting VMs in advance. In this project we built and tested such a system from end to end, using real workload data and an actual CloudSim 5.0 simulation.

We used the PlanetLab CPU traces distributed with CloudSim (11,746 VM-days sampled every five minutes) and compared four machine learning models, namely Ridge Regression, Random Forest, Gradient Boosting and a neural network, against two simple baselines for forecasting CPU demand 10 minutes ahead. Random Forest was selected on validation days and reached an R2 of 0.614 on unseen test days, against 0.293 for the persistence baseline. However, all four models were almost tied, and a plain one-hour moving average was nearly as good (R2 of 0.607), because most short-term variation in these traces is noise. We then wrote a custom CloudSim broker that starts and drains VMs during the simulation and ran 800 simulations. Tuned on validation days, proactive provisioning used 4.7 to 6.9 percent fewer VM hours than a reactive autoscaler and cut scaling events by about 71 percent. It did not, however, hold the 1 percent SLA limit on the test days (1.59 to 2.69 percent, against 0.87 percent for reactive), mainly because one test day was harder to forecast than the days used for tuning. A Streamlit dashboard presents the workload patterns, predictions, provisioning decisions and resource utilization.

1. Introduction

1.1 Background

Elasticity is one of the main promises of cloud computing: resources should grow and shrink with demand, and users should pay only for what they use. In practice this is handled by an autoscaler, which watches a metric such as CPU utilization and changes the number of VMs. Most deployed autoscalers are reactive, meaning they respond to load that has already been measured [9].

A reactive decision takes time to have any effect. The load must first be measured over an interval, and a new VM must then boot and start its application, which can take a few minutes. If demand rises during this time, the running VMs are overloaded until the new one is ready. Predictive autoscaling tries to avoid this by forecasting demand far enough ahead to cover the delay [8].

1.2 Motivation

Provisioning has two opposite costs. Too few VMs means slow responses and broken service level agreements (SLAs), while too many means paying for idle machines. Keeping a fixed pool large enough for the daily peak is simple but wasteful, because the pool is then under-used for most of the day. A good forecast could keep the pool small without being caught out

by rising demand, so it is worth finding out how much a forecast really helps on real workload data, and not only how accurate it looks in a table of prediction errors.

1.3 Problem Being Addressed

A forecasting model with low error does not automatically produce better provisioning. The error that matters is the error at the time and scale at which the scaling decision is made, and an under-prediction costs more than an over-prediction. This project therefore connects the two halves of the problem. We train and compare forecasting models on real traces, then feed the selected model into a running CloudSim simulation and measure the effect on cost, SLA violations, response time and utilization.

2. Problem Statement and Objectives

2.1 Problem Statement

Question 10 of the assignment sheet [11] states: Develop a machine learning application to forecast future virtual machine resource requirements in a cloud data centre. Use a suitable workload dataset and compare the performance of at least three machine learning models for predicting CPU and/or memory requirements. Evaluate the models using appropriate prediction metrics. Integrate the selected model with a cloud simulation environment to demonstrate proactive VM provisioning based on predicted demand. Develop a dashboard to visualize workload patterns, predictions, VM provisioning decisions, and resource utilization.

2.2 Objectives

- To prepare a real VM workload dataset and turn it into a supervised forecasting problem with no leakage from the future.
- To compare at least three machine learning models, and simple baselines, using suitable prediction metrics.
- To select one model using validation data only, keeping the test days for the final evaluation.
- To integrate the selected model with CloudSim through a custom broker that starts and drains VMs during the simulation.
- To compare proactive provisioning with static and reactive provisioning on cost, SLA violations, response time and utilization.
- To build a dashboard showing workload patterns, predictions, provisioning decisions and resource utilization.

3. Dataset and Problem Formulation

3.1 Dataset

We used the PlanetLab workload traces, a set of real CPU utilization measurements collected by the CoMon monitoring project [2] and prepared for cloud research by Beloglazov and Buyya

[1]. They are distributed inside CloudSim itself, which makes them a natural choice for this project. We first tried the Bitbrains GWA-T-12 traces, which also contain memory, but the download server could not be reached. The PlanetLab traces hold CPU only, and the question allows CPU and/or memory, so this project forecasts CPU.

Item	Details
Dataset name	PlanetLab CoMon CPU traces, as distributed with CloudSim
Source	CoMon project [2]; prepared by Beloglazov and Buyya [1]
URL	github.com/Cloudslab/cloudsim, path modules/cloudsim-examples/src/main/resources/workload/planetlab
Number of instances	11,746 VM-days, 3,382,848 samples
Time span	10 non-consecutive days, March and April 2011
Sampling	One sample every 5 minutes, 288 per VM per day
Raw field	CPU utilization of one VM, integer percent (0 to 99)
Engineered features	24 (Table 3)
Target variable	CPU utilization 10 minutes ahead (percent)

Table 1: Dataset summary

Over all days the mean CPU utilization is 11.1 percent, the median is 5 percent and the 95th percentile is 43 percent, while 16.2 percent of samples are exactly zero. Most VMs are lightly loaded and a few are busy, as Figure 1 shows. The most important property for this project is how noisy each VM is: within one VM on a test day, the correlation between one sample and the next averages only 0.09. A VM's recent level is informative, but its next move is close to random.

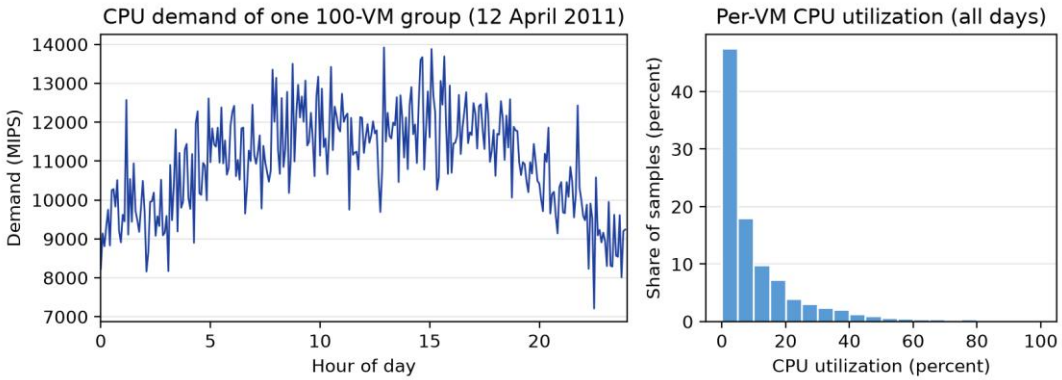


Figure 1: Workload patterns: demand of one VM group over a day, and the distribution of CPU utilization

3.2 Preprocessing and Data Split

Every file was checked to contain exactly 288 values in the valid range. There were no missing values, so no cleaning or imputation was needed, and tree models need no scaling (Ridge and the neural network use a standard scaler fitted on training data). The split follows time, so the

model is always tested on days later than those it learned from. Each VM-day gives 275 rows, because the first hour supplies history and the last 10 minutes have no target.

Split	Days	VM-days	Rows	Use
Training	3, 6, 9, 22, 25 March; 3 April	7,068	1,943,700	Fit the models
Validation	9 and 11 April	2,591	712,525	Select model, margin and targets
Test	12 and 20 April	2,087	573,925	Final results only

Table 2: Time-ordered data split

Each model was trained on the same random sample of 400,000 training rows, repeated for five seeds, so that the slower models were not treated differently. VM names can repeat across days (for example 778 VMs appear on both 11 and 12 April), so a test VM may have been seen in earlier days, though never at a later time.

3.3 Forecasting Problem

Let y_v(t) be the CPU utilization of VM v in interval t. Using only data up to interval t, the model predicts the value two intervals later. One global model is trained on all VMs, which lets it learn general behaviour and apply it to VMs with little history:

y_v(t + 2) = f(y_v(t), y_v(t - 1), ..., y_v(t - 11), fleet state, time of day)

Feature group	Features	Count
Lagged values	CPU at t, t - 1, ..., t - 11 (last hour)	12
Rolling statistics	Mean over 3, 6 and 12 samples; standard deviation over 6 and 12; maximum and minimum over 12; short-term trend	8
Fleet state	Mean CPU of all VMs, now and over the last 6 samples	2
Time of day	Sine and cosine of the time of day	2

Table 3: Input features

3.4 Decision Timing

The forecast horizon comes from the provisioning delay, not from a free choice. The broker makes a decision at the start of each 5 minute interval t. At that moment the latest complete measurement is for interval t - 1. A VM requested now needs 4 minutes to boot and so serves from interval t + 1. Each decision must therefore target interval t + 1 using data up to t - 1, which is two intervals, or 10 minutes, ahead.

3.5 Provisioning Problem and Assumptions

We modelled one service whose load equals the combined demand of a random group of 100 traced VMs, served by a pool of identical worker VMs. Group demand in MIPS (million instructions per second) is D(t) = sum over the group of (y_v(t) / 100) x 1000. The decision variable is the number of VMs n(t) in each interval. The goal is to minimize billed VM hours

while keeping SLA violations small, subject to a boot delay, a maximum pool size, and capacity limits on the hosts. The assignment leaves these details open, so we made the assumptions in Table 4.

Assumption	Value	Justification
Traced VM size	1000 MIPS	Traces give percent only; 1000 MIPS is a VM size used in CloudSim's own PlanetLab examples
VM group size	100 VMs	Large enough to need many workers, small enough that demand still varies (peak about 1.3 times the mean)
Worker VM	1000 MIPS, 1 core, 2 GB RAM, 1 Gbps	A small, common VM size, giving 10 to 20 workers
Hosts	5 hosts, 16 cores, 32 GB RAM, 10 Gbps	Same size as PM1 in Question 7; room for 80 workers, so hosts are never the limit
VM boot delay	4 minutes	Covers VM start and application warm-up; ready by the next interval
Scale-in rule	Remove a VM only after 3 ticks (15 minutes) below the pool size	Avoids removing VMs on one low sample; same for all policies
SLA	One minute of work finishes within 60 s (+1 s for rounding); limit of 1 percent late slices	The pool must keep up with arriving work; 1 percent is a common style of SLA target

Table 4: Simulation assumptions

3.6 Performance Measures

Forecasts are judged at two levels. Per-VM error is the standard measure, but provisioning sees only group demand, so we also report the error of the 100-VM total. For actual value y, prediction p and n samples:

- MAE = (1/n) x sum of |p - y|. Average size of the error in percent CPU.
- RMSE = square root of (1/n) x sum of (p - y)^2. Punishes large errors more.
- R2 = 1 - (sum of (p - y)^2) / (sum of (y - mean of y)^2). Share of variance explained.
- sMAPE = (100/n) x sum of 2|p - y| / (|y| + |p|). Scale-free, and usable when y is zero (the denominator is floored at 1).
- Group MAPE = mean of |P - Y| / Y for group totals P and Y, over 20 random 100-VM groups per day.

Provisioning is judged by the following measures.

- Billed VM hours: time from each VM request to its removal, including boot time, because clouds bill from the start. This is our measure of cost.
- SLA violations: percentage of one-minute work slices that finished late.

- Response time: time from a slice's arrival to its completion, as a mean and at the 99th percentile.
- Pool utilization: completed work divided by the capacity of ready VMs.
- Scaling events: the number of ticks at which VMs were added or removed.

4. System Architecture and Methodology

4.1 Architecture and Workflow

Figure 2 shows the system. The Python side turns traces into features, trains and compares the models, and exports forecasts. The Java side runs the forecasts through CloudSim, where the custom broker makes a scaling decision every five minutes. The results go to the dashboard. Forecasts are computed in advance and read by the broker at the right interval, and we checked that each forecast uses only data available at its decision time.

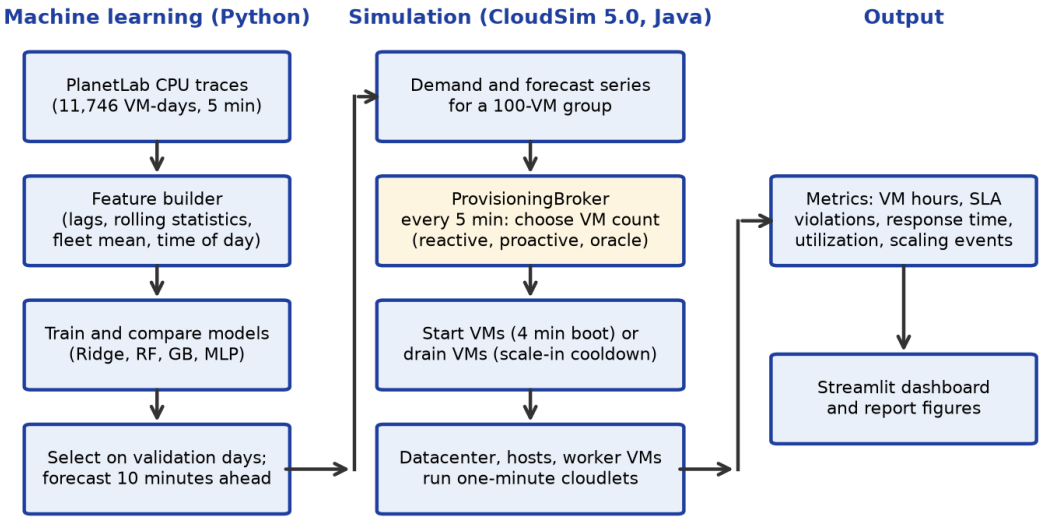


Figure 2: System architecture and workflow

4.2 Forecasting Models

We compared four machine learning models, chosen to cover linear, bagged tree, boosted tree and neural approaches, with two baselines that any useful model must beat.

Model	Configuration
Persistence (baseline)	Predict the latest value
Moving Average (baseline)	Predict the mean of the last 12 samples (one hour)
Ridge Regression	Standardized inputs, alpha = 1.0
Random Forest [5]	100 trees, depth 14, at least 20 samples per leaf, 50 percent of features per split
Gradient Boosting [6]	Histogram method, 300 trees, learning rate 0.05, 31 leaves, at least 50 samples per leaf
Neural Network (MLP)	Two hidden layers (64, 32), batch 1024, up to 30 epochs, early stopping

Table 5: Models compared

All models were built with scikit-learn [4] and use one chosen configuration, without a hyperparameter search. The configurations are sensible standard choices, and since the models ended up tied, a search would be unlikely to change the conclusion. The model with the lowest validation RMSE is selected.

4.3 Provisioning Policies

At each tick a policy estimates the demand D for its target interval, and the pool size follows from a target utilization U (the same formula for every policy):

n(t + 1) = ceiling(D / (1000 x U)), limited to between 1 and 64 VMs

Policy	Demand estimate D	Purpose
Static (peak)	Highest demand of the whole day	Baseline with no scaling; assumes hindsight
Reactive	Latest measurement D(t - 1)	Target tracking, as in common cloud autoscalers
Proactive ML	Forecast of D(t + 1)	The approach under test
Proactive ML + margin	Forecast x 1.143	Protects against under-prediction
Oracle	True D(t + 1)	Best any forecast could do

Table 6: Provisioning policies

The margin of 14.3 percent is the 95th percentile of actual divided by forecast group demand on the validation days, minus one. It is a simple way of learning a safety buffer from past errors. Each policy gets its own target U, because comparing policies at the same U would be misleading: the same U gives different costs. For each policy we chose the cheapest U in the set 0.60, 0.65, ..., 0.95 that kept SLA violations at or below 1 percent on the validation days, and then applied that choice to the test days.

4.4 Dashboard

The dashboard is built with Streamlit and Plotly [10]. The user picks a test day, a VM group, a target utilization and the policies to compare, and all charts update. It has five tabs: workload patterns, predictions, VM provisioning (VM counts, capacity against demand, and a scaling log), resource utilization (CPU, memory and response time), and policy comparison.

5. Mapping the Model to CloudSim

The simulation is a real CloudSim 5.0 [3, 7] program, run on JDK 8 with the setup from Lab 1. Table 7 shows how each element maps to a CloudSim entity.

Problem element	CloudSim entity	Setting
Data centre	Datacenter, Host, Pe	5 hosts of Table 4, VmAllocationPolicySimple
Worker VM	Vm	1000 MIPS, CloudletSchedulerTimeShared
Demand	Cloudlet	One minute of demand, split equally over ready VMs
Autoscaler	ProvisioningBroker (extends DatacenterBroker)	Self-sent tick event every 300 s
VM boot	VM_CREATE_ACK sent with a delay	Request delayed by 240 s
Scale-in	VM_DESTROY after draining	No new work; removed when its cloudlets finish
Overload	Time-shared queue on the Vm	Cloudlets overlap and finish late

Table 7: Correspondence between the problem and CloudSim

The default broker creates all VMs at the start and runs a fixed list of cloudlets, so it cannot do what this project needs. Our broker overrides that behaviour. It schedules events to itself every 60 seconds to release work, and every 300 seconds to take a scaling decision. Since overload is not calculated by a formula but arises from cloudlets sharing a VM inside CloudSim, a late response is a direct result of the simulation. The core logic is in the Appendix, and the full code is in the project repository [12].

6. Experimental Setup

Item	Details
Hardware	Intel Core i7-14700HX (28 logical processors), 16 GB RAM
Operating system	Windows 11
Machine learning	Python 3.13, scikit-learn 1.9, pandas 3.1, NumPy 2.5
Simulation	CloudSim 5.0, Java 8 (Temurin 8u502), Eclipse 2023-12
Dashboard	Streamlit 1.66, Plotly 7.1, Matplotlib 3.11
Forecast runs	4 models x 5 seeds; baselines are deterministic
Test workloads	2 test days x 5 random 100-VM groups = 10
Validation workloads	2 validation days x 5 random 100-VM groups = 10
Simulation runs	10 workloads x 5 policies x 8 targets = 400 for test, and 400 for validation
Simulated period	1 hour 5 minutes to 23 hours 55 minutes of each day (274 intervals)
Forecast metrics	MAE, RMSE, R2, sMAPE, Group MAPE
Provisioning metrics	VM hours, SLA violations, response time, utilization, scaling events

Table 8: Experimental configuration

All policies received exactly the same demand in each workload. CloudSim is deterministic given its input, so repeating a run gives identical results, and the spread in our results comes from the different workloads. The question does not ask for cloud deployment, so the dashboard runs locally.

7. Results

7.1 Forecasting Results

Table 9 gives the results on the test days. The four machine learning rows are averages over five seeds.

Model	MAE	RMSE	R2	sMAPE	Group MAPE
Persistence (baseline)	6.701	12.791	0.293	78.98 %	9.88 %
Moving Average (baseline)	5.351	9.532	0.607	71.83 %	7.55 %
Ridge Regression	5.244	9.467	0.612	71.46 %	7.42 %
Random Forest	5.224	9.446	0.614	71.09 %	7.44 %
Gradient Boosting	5.238	9.444	0.614	71.40 %	7.44 %
Neural Network (MLP)	5.253	9.465	0.613	71.44 %	7.56 %

Table 9: Forecast accuracy on the test days, 10 minutes ahead (MAE and RMSE in percent CPU)

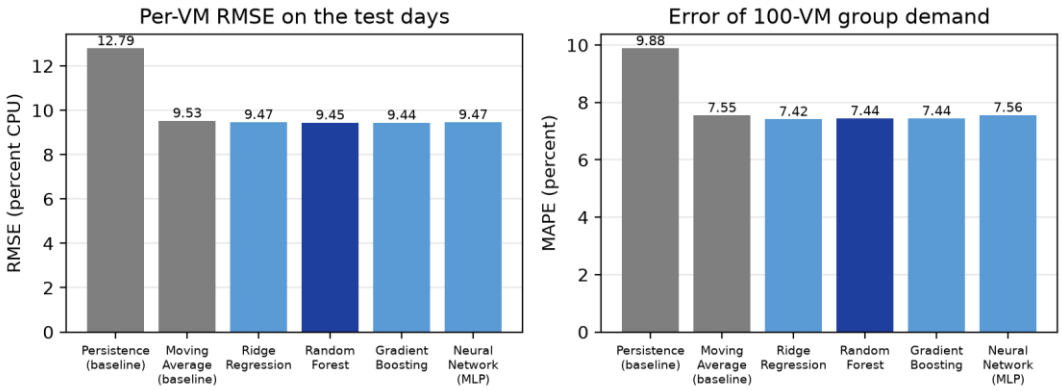


Figure 3: Forecast error of each model on the test days

Random Forest had the lowest validation RMSE and was selected, as Table 10 shows. Its lead is very small: the four validation RMSE values all lie within 0.015 of each other. On the test days Gradient Boosting was in fact slightly better on RMSE (9.444 against 9.446), which we report for honesty but did not use to change the selection. Gradient Boosting also trained about 14 times faster.

Model	Validation RMSE	Validation R2	Training time (s)	Prediction per row (microseconds)
Ridge Regression	8.769	0.663	0.3	0.10
Random Forest	8.754	0.664	33.1	3.45
Gradient Boosting	8.756	0.664	2.4	0.84
Neural Network (MLP)	8.765	0.663	15.9	0.27

Table 10: Model selection on validation days (training on 400,000 rows)

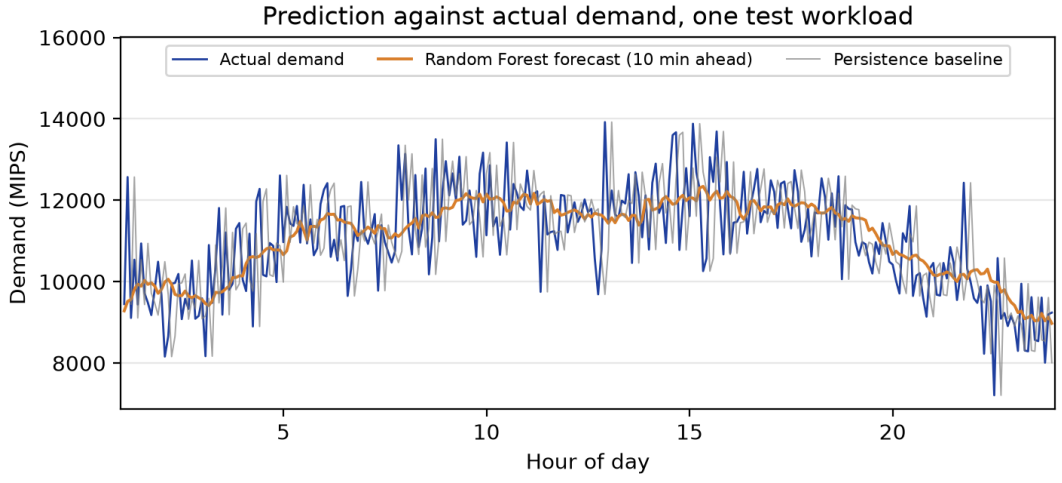


Figure 4: Forecast against actual group demand on one test workload

Figure 4 shows what the numbers mean. The forecast follows the slow daily shape of demand and ignores the rapid swings, while persistence copies each swing two intervals late. Figure 5 shows that accuracy hardly falls with the horizon: Random Forest RMSE rises only from 9.41 at 5 minutes to 9.63 at 60 minutes. The permutation importance of the features agrees: shuffling

the 12-sample mean raises RMSE by 5.61 points, the 6-sample mean by 0.50, and every other feature by less than 0.1.

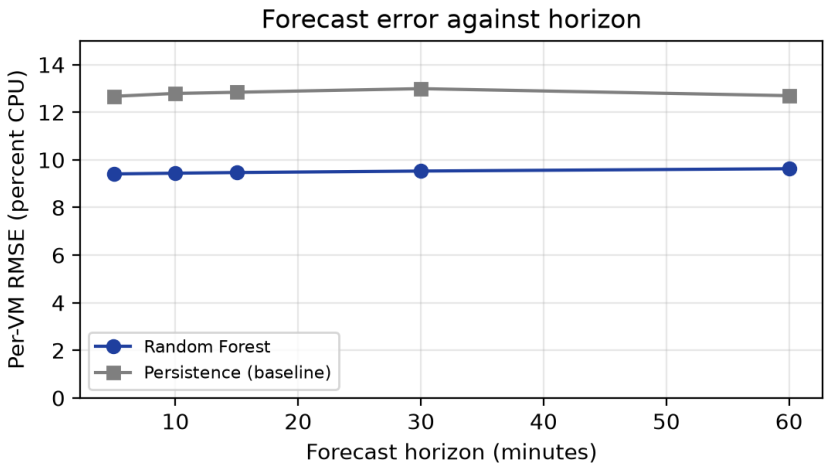


Figure 5: Forecast error against horizon on the test days

7.2 Provisioning Results

Table 11 gives the test results for each policy at the target chosen on the validation days, averaged over 10 workloads. All 800 runs completed all submitted work, and no VM creation failed.

Policy	Target	VM hours	SLA violations	Utilization	P99 response	Scaling events
Static (peak)	0.95	369.8	0.00 %	70.1 %	52.9 s	0.0
Reactive	0.85	338.0	0.87 %	77.7 %	61.7 s	84.3
Proactive ML	0.85	314.9	2.69 %	82.7 %	75.5 s	21.8
ML + margin	0.95	322.5	1.59 %	80.8 %	67.8 s	24.8
Oracle	0.95	304.3	0.05 %	86.2 %	56.9 s	81.1

Table 11: Test results at the validation-tuned targets (average of 10 workloads)

Compared with reactive scaling, Proactive ML used 6.9 percent fewer VM hours and ML + margin 4.7 percent fewer, and both were cheaper on all 10 workloads. They made 74 and 71 percent fewer scaling events. However, neither met the 1 percent limit, and reactive did. The Oracle needed 9.9 percent fewer VM hours than reactive with almost no violations, while the static pool cost 9.7 percent more even with perfect hindsight of the peak.

Table 12 splits the SLA results by day. The two test days behaved very differently.

Policy	12 April	20 April
Reactive	0.74 %	1.00 %
Proactive ML	0.52 %	4.85 %
ML + margin	0.29 %	2.90 %
Oracle	0.10 %	0.00 %

Table 12: SLA violations by test day

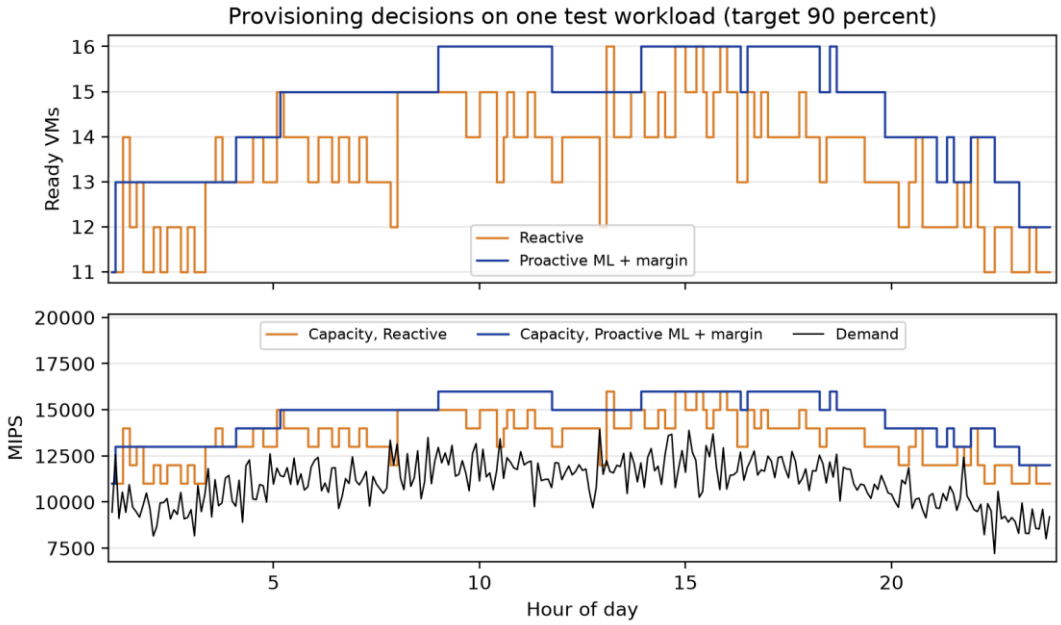


Figure 6: Provisioning decisions on one test workload at a target of 90 percent

Figure 6 shows the main behavioural difference. The reactive pool moves up and down with almost every noisy sample, while the ML + margin pool follows the shape of the day in a few steps. Figure 7 compares all policies across all eight targets, which removes the effect of choosing a target. A curve that is lower or further left is better.

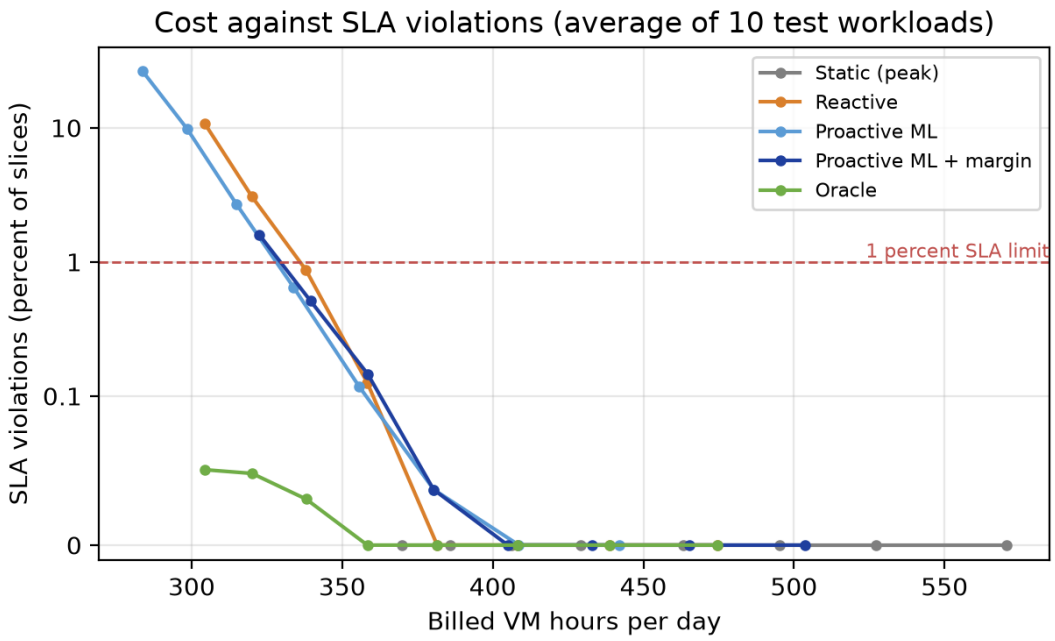


Figure 7: Cost against SLA violations for all targets, with the vertical axis on a log scale above 0.1

ML + margin at a target of U + 0.05 costs about the same as reactive at U, which allows the direct comparison in Table 13.

Reactive target	ML + margin target	VM hours (reactive / ML)	SLA violations (reactive / ML)	Workloads where ML is better
0.80	0.85	358.3 / 358.5	0.13 % / 0.15 %	1 of 10 (6 tied)
0.85	0.90	338.0 / 339.5	0.87 % / 0.51 %	6 of 10 (1 tied)
0.90	0.95	320.0 / 322.5	3.07 % / 1.59 %	8 of 10

Table 13: Reactive and ML + margin at about equal cost

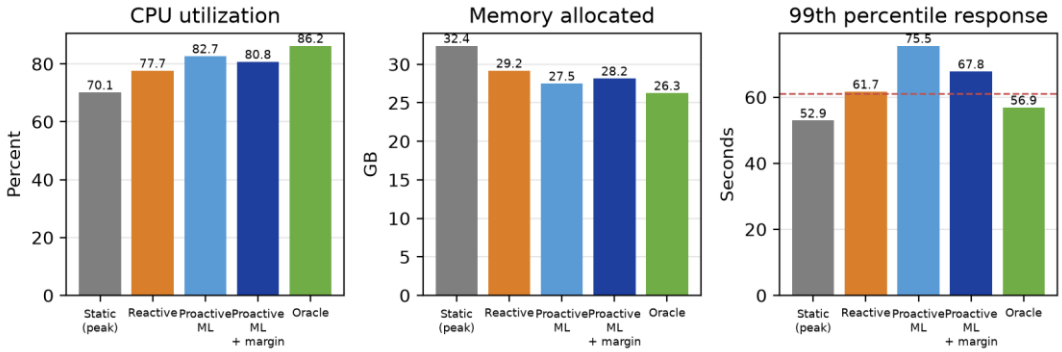


Figure 8: CPU utilization, allocated memory and response time at the tuned targets

7.3 Dashboard

Figures 9 to 13 are screenshots of the running dashboard for one test workload at a target of 80 percent. Every chart has a title, labelled axes with units, and a legend.

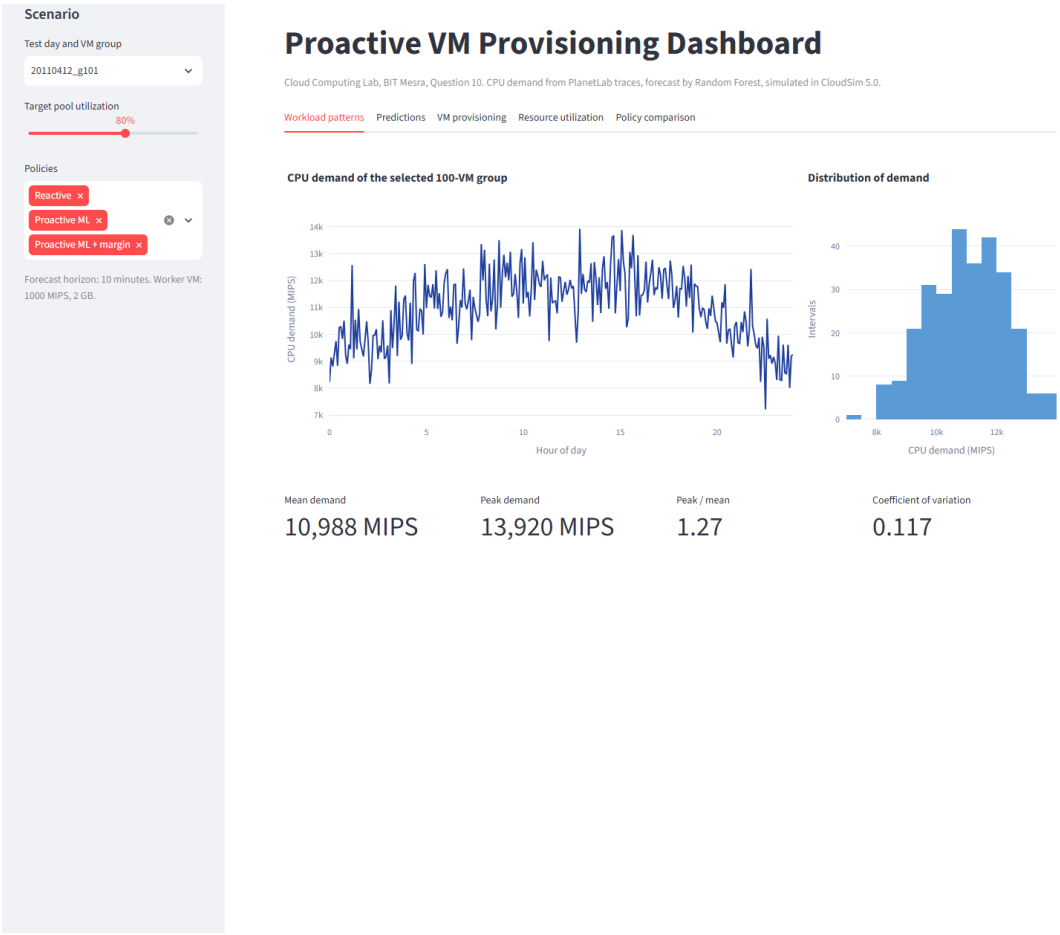


Figure 9: Dashboard, workload patterns

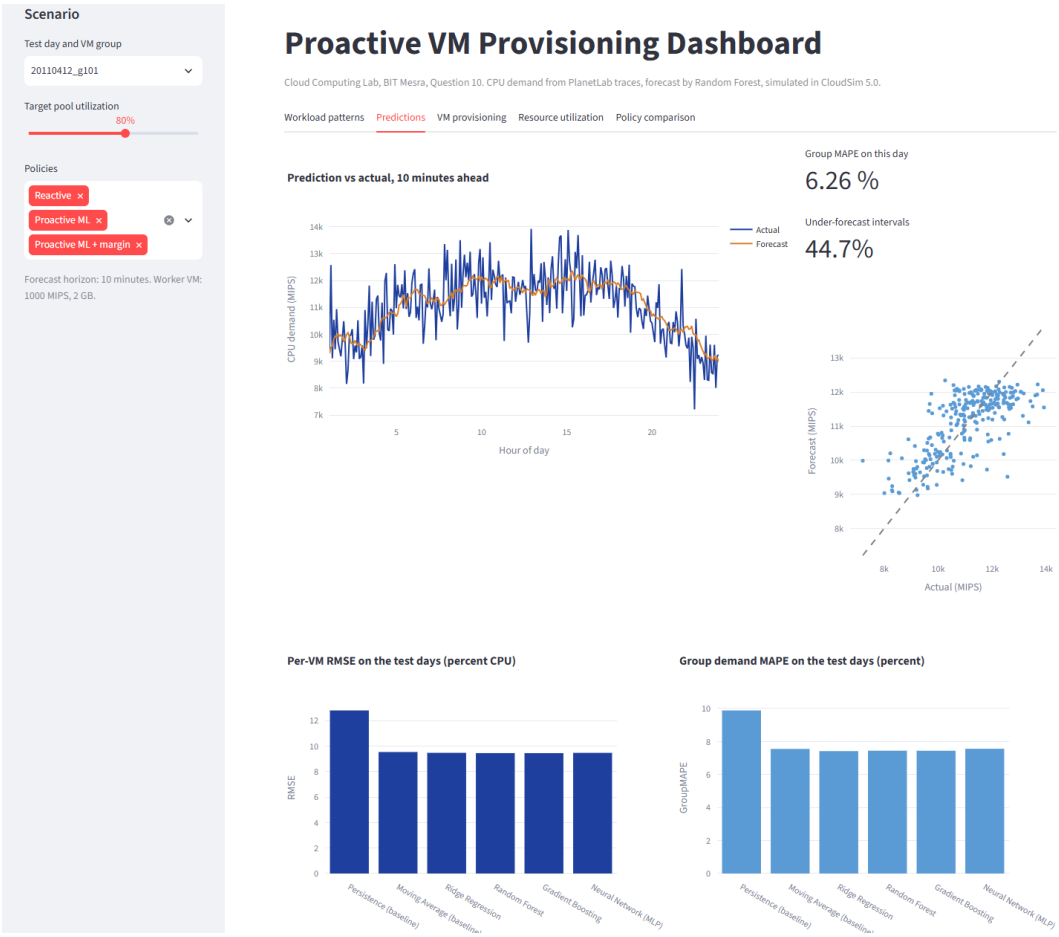


Figure 10: Dashboard, prediction against actual demand

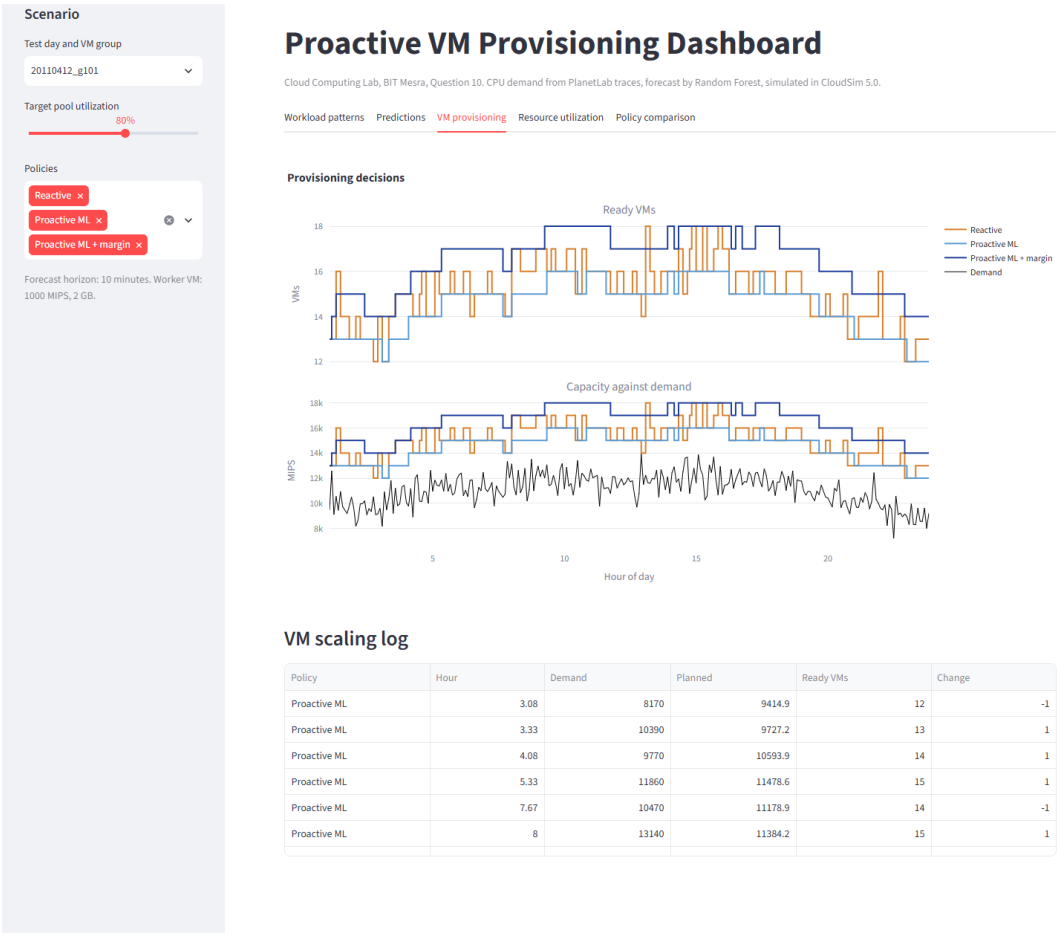


Figure 11: Dashboard, VM provisioning decisions

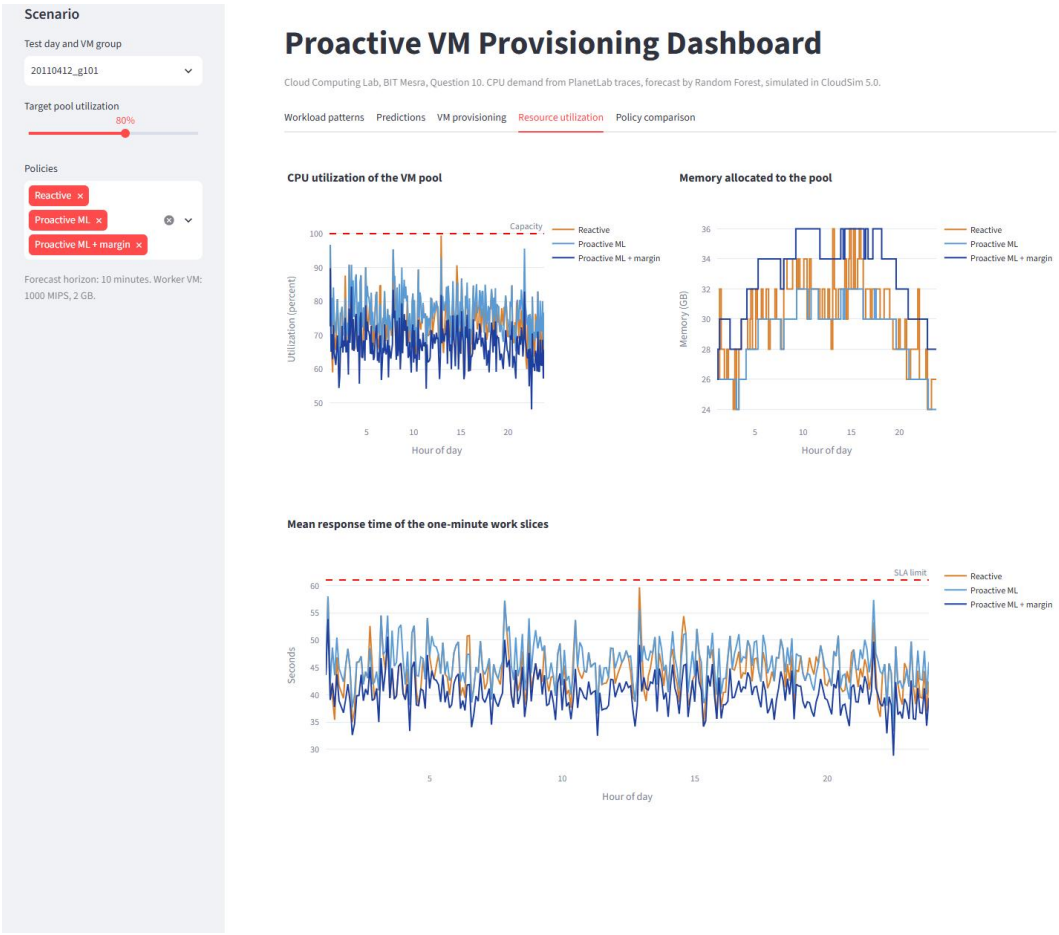


Figure 12: Dashboard, resource utilization

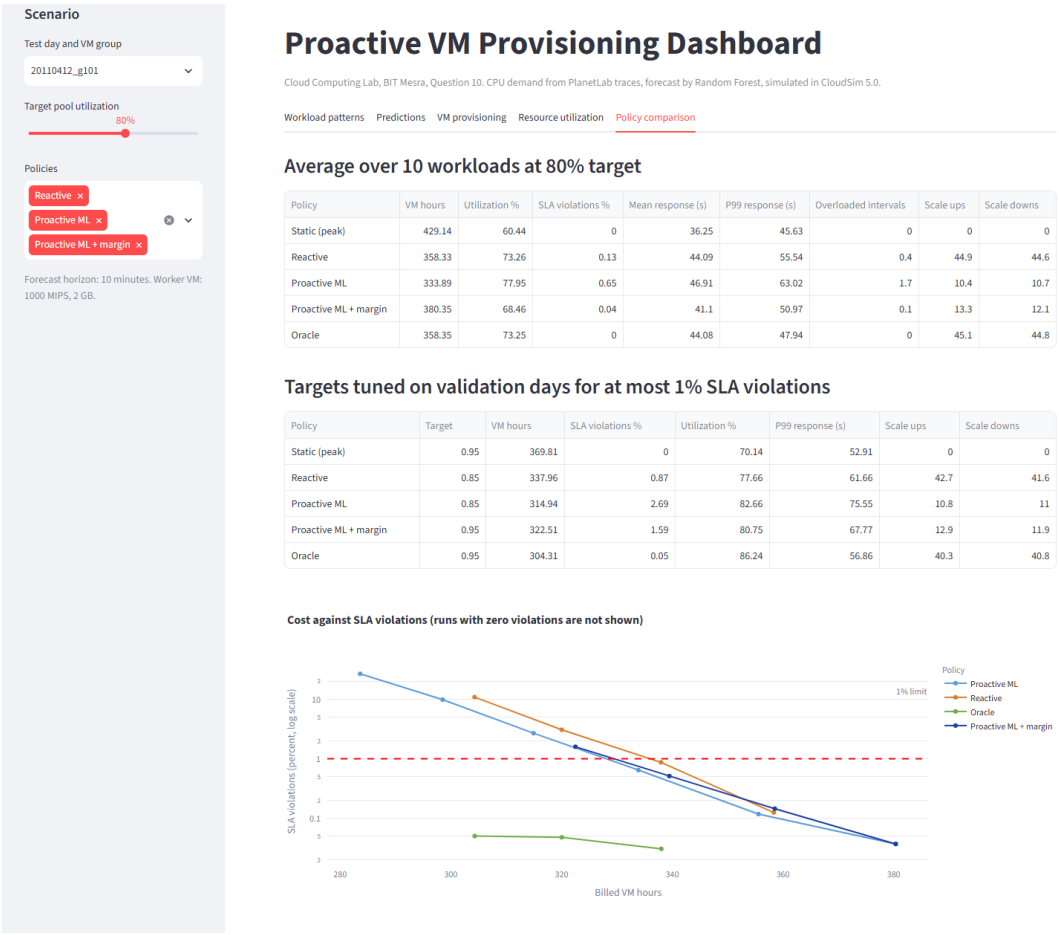


Figure 13: Dashboard, policy comparison

8. Analysis and Discussion

8.1 What Worked

The whole pipeline worked from end to end, on real data and in real CloudSim. All four models beat persistence clearly: RMSE fell from 12.79 to about 9.45, and group MAPE from 9.88 to 7.44 percent. The clearest provisioning benefit is stability, with about 70 to 74 percent fewer scaling events, which means fewer VM starts and less disruption to a service. The forecast also saved money at the same target utilization, and at the low-cost end of Figure 7 it reduced SLA violations for the same spending: Table 13 shows 0.87 falling to 0.51 percent and 3.07 falling to 1.59 percent. On 12 April, ML provisioning beat reactive on both cost and SLA.

8.2 What Did Not

The tuned proactive policies missed the 1 percent SLA limit on the test days, which is the most important negative result of this project. The learned margin did not generalize: on 20 April, violations reached 2.90 percent with the margin and 4.85 percent without it, against 1.00 percent for reactive. The added machine learning also gave little gain over a simple method: the selected model was only slightly better than a one-hour moving average (R2 of 0.614 against 0.607), and the choice of model barely mattered. At high spending the forecast did not help, since Table 13 shows reactive and ML equal at the first row (0.13 and 0.15 percent).

8.3 Why

The models tie because there is little left to learn. Each VM has a level that persists over an hour, but its movement around that level is nearly random (the correlation between neighbouring samples is about 0.09). Almost all the signal is in the recent mean, which every model finds, and which the moving average already is. This is also why accuracy barely changes with the horizon and why persistence does so badly: it copies noise.

The saving in scaling events has the same cause. A reactive policy sees noise as real change, so it scales up on random peaks. The scale-in rule then holds those VMs for 15 minutes, which keeps the pool high. A smooth forecast does not chase these peaks. This same smoothness is a weakness, though: the forecast deliberately misses real short peaks, and the reactive pool, by holding on to recent peaks, happens to carry spare capacity that absorbs later ones. This explains the Oracle's advantage, which comes from knowing the noise in advance and which no forecast of this data can reach.

The failure on 20 April comes from a difference between days. On the validation days and on 12 April the forecast bias was within 0.5 percent and group MAPE was 6.3 to 6.6 percent. On 20 April the forecast was 3.0 percent too low on average, and group MAPE rose to 8.5 percent. The actual-to-forecast ratio reached 1.22 at the 95th percentile, more than the 1.14 margin learned earlier. So a margin taken from a few past days can fail on a new day. The reactive policy does not depend on such a model and so was more robust here.

8.4 Trends Observed

- Forecast accuracy is nearly flat from 5 to 60 minutes ahead, so horizon is not the limit here; the noise is.
- Every policy shows the same trade-off: raising the target lowers cost and raises SLA violations, slowly at first and then sharply. Above a target of about 0.85, violations grow by roughly three to four times for each 0.05 increase in the target.
- At equal targets, the forecast runs hotter than reactive: at 0.85, Proactive ML costs 315 VM hours against 338, with 2.69 against 0.87 percent violations. This is why equal-target comparisons mislead.
- Memory follows the VM count, because each worker holds a fixed 2 GB, so its saving matches the VM saving.
- The gain from forecasting is bounded. Even the Oracle saved only 9.9 percent against reactive here, since this workload peaks at only about 1.3 times its mean. A burstier workload would leave more to gain.

8.5 Limitations

- Only CPU is forecast, because the PlanetLab traces contain no memory data. Memory in our results is allocated memory, not forecast demand.

- Only two test days and ten workloads were used, and the VM groups on a day overlap. The results show a direction, not a precise estimate, and one difficult day strongly affected them.
- The trace days are not consecutive, so weekly patterns and day-to-day continuity cannot be used.
- The MIPS scale, boot delay, scale-in rule and SLA are our assumptions, and different values would change the numbers. The boot delay in particular fixes the 10 minute horizon.
- Traced VMs are treated as parts of one service with evenly divided work. Real services route requests differently, and work is not shifted away from a VM that has fallen behind.
- Each model has one configuration, with no hyperparameter search, and the forecast is trained once and never retrained during a day.
- Hosts are never the limit, so VM placement, migration and energy are not studied.

9. Validation of the Simulation

Because the conclusions depend on the custom broker, we tested it on small scenarios whose correct answer can be worked out by hand. The program SimulatorChecks runs these automatically, and all checks passed. The results are in Table 14.

Check	Expected	Measured
Steady load 1600 MIPS, target 0.8: VMs used	2	2
Same: response time (60 s x 0.8)	48.0 s	48.0 s
Same: SLA violations	0 %	0 %
Same: work completed equals work submitted	8,640,000 MI	8,640,000 MI
Step in demand: reactive VMs before the step	1	1
Step: reactive VMs one boot after seeing it	4	4
Step: oracle VMs when the step begins	4	4
Step: oracle SLA violations	0 %	0 %
Static pool: VMs from the peak	3	3
Static pool: billed hours (3 VMs, 8 intervals)	2.00	1.99
Static pool: utilization equals the target	80 %	80.4 %

Table 14: Simulator checks

The step test confirms the core idea of the project in the simulator. With the same demand step, the reactive policy was late and the Oracle was on time, so the timing of Section 3.4 is what the broker actually does. The small differences in the last two rows arise because the final slice finishes before the end of its interval, so the VMs are released slightly early. We also confirmed

that completed work equals submitted work in all 800 full runs, so no work is lost when VMs are drained.

To protect the results from bias, the model, margin and target were all chosen on validation days, and the test days were used only for final reporting. When the test results turned out worse than validation, we kept them and did not retune.

10. Conclusion

We built a complete proactive provisioning system: a forecasting pipeline on real PlanetLab traces, a custom CloudSim 5.0 broker that scales VMs during the simulation, and an interactive dashboard. Machine learning models clearly beat persistence (R2 of about 0.61 against 0.29), but were almost tied with each other and only slightly better than a moving average. In the simulation, proactive provisioning cut VM hours by about 5 to 7 percent and scaling events by about 70 percent compared with a reactive autoscaler. However, it broke the 1 percent SLA limit on the test days, mainly on one day that was harder to forecast than the tuning days, while the reactive policy held it. Our conclusion is therefore mixed: on this workload, forecasting gives a calmer and somewhat cheaper pool but less safety, and its main value is stability rather than lower SLA violations. The project also shows why such a system must be judged in simulation and not only by prediction error, since near-identical forecast errors hid a large difference between days.

11. Future Scope

- Use a trace with both CPU and memory, such as Bitbrains or the Google cluster data, to forecast both resources and size VMs on both.
- Replace the fixed margin with one that adapts to recent forecast errors during the day, and test whether this holds the SLA on days like 20 April.
- Combine the policies by taking the larger of the forecast and the latest measurement, which may keep the stability of the forecast and the safety of reactive scaling.
- Use quantile regression to predict an upper bound on demand directly, since under-prediction costs more than over-prediction.
- Test on consecutive days with more test days, and retrain the model during the simulation.
- Test burstier workloads and different boot delays, where forecasting should matter more.
- Add host-level consolidation and a power model to measure energy, using CloudSim's power package.

References

[1] A. Beloglazov and R. Buyya, "Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers," Concurrency and Computation: Practice and Experience, vol. 24, no. 13, pp. 1397-1420, 2012.

[2] K. Park and V. S. Pai, "CoMon: a mostly-scalable monitoring system for PlanetLab," ACM SIGOPS Operating Systems Review, vol. 40, no. 1, pp. 65-74, 2006.

[3] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose and R. Buyya, "CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," Software: Practice and Experience, vol. 41, no. 1, pp. 23-50, 2011.

[4] F. Pedregosa et al., "Scikit-learn: machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[5] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[6] J. H. Friedman, "Greedy function approximation: a gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001.

[7] CloudSim project page and documentation, Cloud Computing and Distributed Systems (CLOUDS) Laboratory, University of Melbourne. https://github.com/Cloudslab/cloudsim

[8] N. Roy, A. Dubey and A. Gokhale, "Efficient autoscaling in the cloud using predictive models for workload forecasting," Proceedings of IEEE International Conference on Cloud Computing, pp. 500-507, 2011.

[9] T. Lorido-Botran, J. Miguel-Alonso and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," Journal of Grid Computing, vol. 12, no. 4, pp. 559-592, 2014.

[10] Python, pandas, NumPy, Matplotlib, Streamlit and Plotly documentation (python.org, pandas.pydata.org, numpy.org, matplotlib.org, streamlit.io, plotly.com).

[11] Assignment sheet, Cloud Computing Lab: Day to Day Performance, Question 10, BIT Mesra, September 2026.

[12] Project source code and results. https://github.com/Aditya-XR/proactive-vm-provisioning

Appendix: Core Logic of the Provisioning Broker

The following shortened code from ProvisioningBroker.java runs at every tick. The complete program, the training code and steps to reproduce every result are in the repository [12].

```
double planned = plannedDemand();  // depends on the policy
int target = vmsFor(planned);      // ceil(planned / (mips x U))
recentTargets.addLast(target);
int pool = ready - draining + booting;
if (target > pool) {
    for (int i = pool; i < target; i++) {
        requestVm(bootDelaySeconds);  // VM_CREATE after 240 s
    }
} else if (recentTargets.size() == scaleInTicks) {
    int keep = max(recentTargets);    // last 3 ticks
    if (keep < pool) {
        drain(pool - keep);  // no new work, destroy when idle
    }
}
```